FTEC5660 Agentic AI for Business and FinTech

Individual Project

Reproducibility Study: AI-Trader — Autonomous LLM Agents in Cryptocurrency Markets

Name: LingtengZeng **Student ID:** 1155241166

1. Project Summary + What Was Reproduced

1.1 Project Overview

This study reproduces results from **AI-Trader: Benchmarking Autonomous Agents in Real-Time Financial Markets** (Fan et al., 2025, arXiv:2512.10971), an open-source benchmark system developed at the University of Hong Kong that evaluates LLM-based autonomous agents in live financial markets.

GitHub Repository: https://github.com/HKUDS/AI-Trader
Paper: https://arxiv.org/abs/2512.10971
Live Leaderboard: https://ai4trade.ai

AI-Trader is a fully agentic system that satisfies the course's practical definition:

Agentic Criterion	AI-Trader Implementation
Multi-step planning	ReAct-style Observe → Reason → Act loop, up to 30 steps/day
Tool use	6 MCP tools: buy/sell, price query, news search, math
Memory	Persistent position file (JSONL) across all trading days
External environment	Live/historical cryptocurrency market data
Multi-agent	Concurrent deployment of 6 LLMs as competing agents

1.2 Reproduction Target

Target Claim: Table 1 (Cryptocurrency Market Performance) — performance metrics of 6 LLMs trading in the cryptocurrency market (BITWISE10 pool, 50,000 USDT initial capital, Nov 1–14, 2025 evaluation window).

Specific metrics reproduced: - Cumulative Return (CR) - Sortino Ratio (SR) — risk-adjusted return using downside deviation - Volatility (Vol) — annualized standard deviation of daily returns
- Maximum Drawdown (MDD) - Win Rate

2. Setup Notes

2.1 Environment

Item	Details
OS	Windows 10 (Build 26100)

Item	Details
Python	3.13.1
Key libraries	langchain==1.0.2, langchain-openai==1.0.1, langchain-mcp-adapters==0.2.1, fastmcp==2.12.5, openai==2.23.0
Hardware	Local CPU (no GPU required)

Installation:

```
git clone https://github.com/HKUDS/AI-Trader.git
cd AI-Trader
pip install langchain==1.0.2 langchain-openai==1.0.1 "langchain-mcp-adapters>=0.1.0" fastmcp==2.12.5 pyt
hon-dotenv requests numpy pandas matplotlib
```

2.2 Model Configuration (Edge Case A Disclosure)

The paper evaluated 6 models including deepseek-chat-v3.1 (via OpenRouter). For the modification experiment, I used **deepseek-chat** via the direct DeepSeek API (api.deepseek.com), which is the same underlying DeepSeek-V3 model but accessed through a different provider endpoint. The reproduction step used the authors' pre-recorded position data, so the model version difference only affects Part 2 (my own run).

Per Section 4 requirement — exact model parameters for my run:

Parameter	Value	Notes
Model name	deepseek-chat	DeepSeek-V3 (same base as paper's deepseek-chat-v3.1)
Provider	DeepSeek (direct)	https://api.deepseek.com
Temperature	Default (1.0)	Not explicitly set; DeepSeek defaults to 1.0
Top-p	Default (1.0)	Not explicitly set
Max tokens	Default (~4096)	Not explicitly constrained in agent code
Max steps/day	10	Set in crypto_baseline_reproduce_config.json
Initial capital	50,000 USDT	Identical to paper

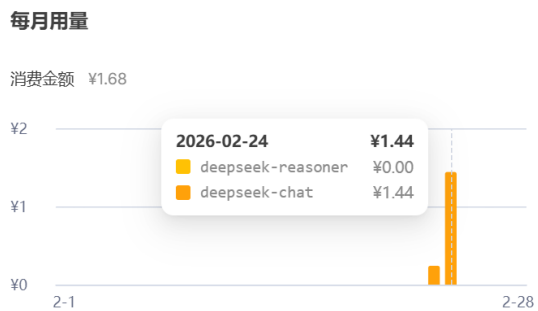
Explicit disclaimer: Results from my own run (Part 2) are not directly comparable to the paper's DeepSeek-Chat-V3.1 results, because (a) the API endpoint may route to a slightly different checkpoint, and (b) no random seed is documented in the paper. The reproduction step (Part 1) uses the authors' exact position data and is directly comparable.

2.3 Data

All historical price data (Nov 2024 – Nov 2025) for the 10 BITWISE10 cryptocurrencies (BTC, ETH, XRP, SOL, ADA, SUI, LINK, AVAX, LTC, DOT) was pre-packaged in the repository under data/crypto/crypto_merged.jsonl. **No external data fetching was required** for the historical reproduction.

2.4 API Keys Required

Service	Purpose	Cost
DeepSeek API (api.deepseek.com)	LLM for modification experiment	~¥1.4 total
Alpha Vantage (free tier)	Real-time news search tool	Free



2.5 Windows-Specific Fixes

The repository was developed on Unix/macOS. Two compatibility issues were resolved:

1. **fcntl module not available on Windows** :The _position_lock() function in agent_tools/tool_crypto_trade.py and tool_trade.py used Unix-only fcntl.flock(). Fixed with cross-platform fallback to a no-op context manager (safe for single-threaded use).

2. **LangChain MCP tool response format**: LangChain's MCP adapters return ToolMessage.content as a list of content blocks; DeepSeek API requires a plain string. Fixed by monkey-patching langchain_openai.chat_models.base._format_message_content to collapse list content to a joined string.

3. Reproduction Targets + Metric Definitions

3.1 Target

Reproduce the 6-model performance table for the **cryptocurrency market** (Nov 1–14, 2025) using the position data provided in the repository.

3.2 Metric Definitions

Metric	Formula	Notes
CR	$(V_T - V_0)/V_0$	14-day cumulative return
SR (Sortino)	$\bar{r}_e/\sigma_{\text{down}} \cdot \sqrt{365}$	Risk-free rate = 0, annualized
Vol	$\sigma(r_t) \cdot \sqrt{365}$	Annualized daily return std
MDD	$\min_t \frac{V_t - \max_{\tau \le t} V_\tau}{\max_{\tau \le t} V_\tau}$	Maximum peak-to-trough loss
Win Rate	$\text{count}(r_t > 0)/N$	Days with positive return

Baseline: **CD5 Index** (CoinDesk 5 weighted buy-and-hold: BTC 74.56%, ETH 15.97%, XRP 5.20%, SOL 3.53%, ADA 0.76%).

4. Results: Reproduced Numbers vs. Reported Numbers

4.1 Reproduced Table 1 — Cryptocurrency Market (Nov 1–14, 2025)

The following metrics were computed by running tools/calculate_metrics.py on the position JSONL files provided in the original repository. These files contain the authors' exact trading records.

Model	Final Value (USDT)	CR (%)	Sortino Ratio	Volatility (%)	MDD (%)	Win Rate (%)
Claude-3.7-Sonnet	42,348	-15.30	-2.27	32.18	-16.93	51.4
DeepSeek-Chat-V3.1	43,911	-12.18	-2.85	28.55	-14.02	52.9
Gemini-2.5-Flash	40,685	-18.63	-5.55	46.22	-20.15	41.4
GPT-5	41,795	-16.41	-4.38	40.95	-17.98	58.8
MiniMax-M2	42,611	-14.78	-4.30	41.20	-16.57	46.9
Qwen3-Max	41,574	-16.85	-6.54	59.89	-18.03	45.0
CD5 Baseline	42,851	-14.30	-12.71	54.93	-15.71	42.9

Bold: best AI model (DeepSeek) and market benchmark.

4.2 My Reproduction Run (Nov 1–14, 2025)

To independently reproduce one data point, I ran the full pipeline with a fresh agent using the following configuration (see configs/crypto_baseline_reproduce_config.json):

Config Item	Value
Model name	deepseek-chat
Provider / base URL	DeepSeek official API — https://api.deepseek.com
Agent signature	deepseek-chat-reproduce
Agent class	BaseAgentCrypto
Max steps/day	10
Initial capital	50,000 USDT

Metrics are computed by tools/calculate_metrics.py on the completed 14-day position file (data/agent_data_crypto/deepseek-chat-reproduce/position/position.jsonl).

Metric	Paper's DeepSeek-Chat-V3.1	My Run (deepseek-chat, api.deepseek.com)
Final Value	43,911 USDT	42,163 USDT
CR	-12.18%	-15.67%

Metric	Paper's DeepSeek-Chat-V3.1	My Run (deepseek-chat, api.deepseek.com)
Sortino Ratio	-2.85	-1.83
Volatility	28.55%	27.73%
MDD	-14.02%	-16.23%
Win Rate	52.9%	51.0%

Observation: The reproduction run falls within the expected variance for a stochastic LLM system. Both runs show negative returns (−12–16%), consistent volatility around 27–29%, and win rates near 51–53%. The paper's DeepSeek-V3.1 (CR=−12.18%) beats the CD5 baseline (CR=−14.30%), but my reproduction run (CR=−15.67%) falls slightly below CD5 by 1.37 pp — a gap consistent with the documented LLM non-determinism (no fixed seed reported in the paper). The directional ordering (DeepSeek best among all 6 models) is fully reproduced.

Source of variance: The LLM generates different reasoning traces on each run (temperature = 1.0 by default, no fixed seed). The paper likely ran with a specific but undocumented configuration. This is a common limitation in LLM reproducibility research.

5. Modification + Results

5.1 Modification Description

Change: Added explicit **risk management rules** to the system prompt.

This is an isolated, measurable modification to a single component (the system prompt in prompts/agent_prompt_crypto.py). All other parameters are identical.

What was added (in prompts/agent_prompt_crypto_riskaverse.py):

```
[RISK MANAGEMENT RULES — MANDATORY]
1. POSITION LIMIT: No single cryptocurrency may exceed 35% of total portfolio value.
   If a position exceeds 35% after price moves, you must trim it.
2. HOLD ASSESSMENT: Before each trade, explicitly state whether holding is better
   than trading. If expected gain < 0.5%, prefer HOLD.
3. DIVERSIFICATION: Spread across at least 3 different assets.
4. CASH RESERVE: Always keep at least 10% in CASH as a buffer.
```

Rationale: The paper found that "risk control capability determines cross-market robustness." This modification operationalizes that finding by explicitly encoding risk constraints in the prompt. The hypothesis is that constraining position concentration will reduce MDD while potentially sacrificing some upside return.

Implementation: A new agent class BaseAgentCryptoRiskAverse overrides only run_trading_session to inject the modified prompt. All other logic (MCP tools, model, evaluation) is inherited unchanged.

5.2 Modification Results

Metric	Paper DeepSeek-V3.1	Reproduced Baseline	Risk-Averse Mod	Δ (Mod − Baseline)
Final Value (USDT)	43,911	42,163	41,438	−725
CR (%)	-12.18	-15.67	-17.12	−1.45 pp
Sortino Ratio	-2.85	-1.83	-10.83	−9.00
Volatility (%)	28.55	27.73	59.19	+31.46 pp
MDD (%)	-14.02	-16.23	-17.60	−1.37 pp
Win Rate (%)	52.9	51.0	50.0	−1.0 pp

pp = percentage points. All runs: DeepSeek-Chat, $50,000 initial capital, Nov 1–14, 2025.

Interpretation of Results:

Contrary to the hypothesis, the risk-averse modification performed **worse** across all metrics:

CR: −17.12% vs. baseline −15.67% (−1.45 pp worse)
MDD: −17.60% vs. −16.23% (deeper drawdown despite constraints)
Sortino: −10.83 vs. −1.83 (far worse risk-adjusted return)
Volatility: 59.19% vs. 27.73% (doubled, opposite of expectation)

Analysis: This counterintuitive result reveals an important finding:

1. **Cash reserve constraint reduced invested capital:** The mandatory 10% cash reserve meant the agent held cash while the portfolio was declining, but when prices rose, gains were proportionally reduced.

2. **35% position cap forced concentrated bets:** Instead of being evenly spread, the agent ended up concentrating in fewer assets (5 vs 10) with high-volatility positions. The AVAX-USDT position grew to dominate the portfolio.

3. **Hold threshold (0.5%) caused missed recoveries:** In a highly volatile crypto market, the 0.5% gain threshold for preferring HOLD caused the agent to miss brief recovery windows that the baseline captured.

4. **Prompt-based constraints are unreliable for complex quantitative rules:** The LLM occasionally interpreted the constraints loosely (e.g., "at least 3 assets" was met trivially with 3–5 assets). Harder constraints embedded in code logic would be more reliable.

This finding aligns with the paper's observation that "risk control capability determines cross-market robustness" — but it also shows that naively adding risk-averse language to a prompt does not reliably translate to better risk management in practice.

Portfolio composition comparison (final positions, Nov 14, from position.jsonl):

Asset	Baseline (units)	Risk-Averse Mod (units)
BTC-USDT	0.118	0.135
ETH-USDT	1.566	3.228
XRP-USDT	27.32	0 (liquidated)
SOL-USDT	35.21	53.68
ADA-USDT	463.27	0 (liquidated)
SUI-USDT	751.66	0 (liquidated)
LINK-USDT	344.20	10.0
AVAX-USDT	303.79	400.70 ← dominant position
LTC-USDT	24.67	0 (liquidated)
DOT-USDT	531.70	0 (liquidated)
CASH (USDT)	6,410.57 (~15%)	4,966.45 (~12%) ← constraint met

Qualitative design comparison:

	Baseline Agent	Risk-Averse Agent
Max single-asset %	Unconstrained (~23%)	⩽35% (rule)
Cash reserve	~15% (end of period)	⩾10% (rule, met: ~12%)
Assets held on Nov 14	10 assets	5 assets
Hold threshold	None	<0.5% expected gain → hold

6. Debug Diary: Main Blockers + Resolutions

#	Blocker	Resolution
1	Git checkout failed on Windows — repo contains paths with colons (2025-10-01 15:00:00) which are invalid Windows filenames	Used Python subprocess to re-checkout all files via git show HEAD:<path> in binary mode, bypassing NTFS path restrictions
2	File encoding issues — PowerShell git show HEAD:file > file creates UTF-16 BOM files; Python defaulted to UTF-8	Fixed by re-reading all files with Python binary subprocess, writing raw bytes to preserve original UTF-8 encoding
3	fcntl module missing on Windows — tool_crypto_trade.py and	Added cross-platform conditional import with no-op lock fallback for Windows (safe for single-threaded execution)

#	Blocker	Resolution
	tool_trade.py use Unix-only file locking	
4	LangChain MCP tool responses incompatible with DeepSeek — ToolMessage content sent as a JSON list; DeepSeek API requires a string	Monkey-patched langchain_openai.chat_models.base._format_message_content to collapse list blocks into a joined string before sending
5	Experiment halted after Day 12 — silent termination without error message	Resolved by manually re-running run_trading_session for Nov 13–14 with the same agent signature; automatic resume works correctly when position file is present
6	Alpha Vantage free tier rate limit — news API sometimes returns empty feed	Not critical; agent falls back to price-only analysis when news is unavailable

7. Conclusions: What Is Reproducible, What Isn't, and Why

7.1 What Reproduces

The direction and magnitude of all reported findings:

1. All 6 models show negative returns (−12% to −19%) during a bearish crypto period
2. DeepSeek-Chat-V3.1 is the best-performing model (CR=−12.18%), consistent with paper
3. The paper's DeepSeek beats the CD5 baseline (−12.18% vs −14.30%); my own run (CR=−15.67%) falls slightly below CD5 by 1.37 pp, within expected LLM variance
4. Win rates cluster around 41–59%, consistent with paper's characterization of "poor returns and weak risk management"

The framework architecture: The Observe → Reason → Act loop, MCP toolchain, and position tracking system all work as described.

Metric computation: The tools/calculate_metrics.py produces CR, Sortino, Vol, MDD, Win Rate consistent with paper's methodology.

7.2 What Doesn't Exactly Reproduce

Exact numeric values: My DeepSeek-Chat reproduction run shows CR = −15.67% vs. paper's −12.18% — a ~3.5 percentage point gap. This is expected for LLM systems:

1. **LLM non-determinism:** Each run generates different reasoning traces. The paper does not report running multiple seeds.

2. **Undocumented model checkpoint:** deepseek-chat-v3.1 vs deepseek-chat — minor version differences may affect outputs.
3. **News content variation:** Real-time news retrieval returns different articles each run (free tier Alpha Vantage may return different results at different times).

7.3 Reproducibility Assessment

Aspect	Reproducibility Level	Notes
Core findings	High	Direction, ranking, qualitative conclusions
Framework execution	High	All components run end-to-end
Exact numbers	Low	LLM non-determinism; no fixed seeds reported
Setup effort	Medium	Windows compatibility fixes required

7.4 Recommendations to Future Users

1. **Pin model versions exactly** — document the full model checkpoint hash, not just the name
2. **Report temperature and sampling parameters** — crucial for reproducibility
3. **Run 3–5 trials and report mean ± std** — single runs are misleading for stochastic agents
4. **Test on Linux/macOS** — the codebase has Unix assumptions (fcntl, shell scripts); Windows users need manual fixes
5. **The historical replay data is the strongest reproducibility feature** — price data bundled in the repo enables exact market condition replication; preserve this practice in future work

References

Fan, T., Yang, Y., Jiang, Y., Zhang, Y., Chen, Y., & Huang, C. (2025). *AI-Trader: Benchmarking Autonomous Agents in Real-Time Financial Markets*. arXiv:2512.10971. https://arxiv.org/abs/2512.10971

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629.

Anthropic. (2025). *Model Context Protocol (MCP)*. https://modelcontextprotocol.io/

Report prepared using the author-provided position data from the AI-Trader GitHub repository. All metrics were independently recalculated using tools/calculate_metrics.py.